

# Ekalivan — App Functionality & Executive Summary

---

கற்போம் • கேட்போம் • பார்ப்போம் • பழகுவோம் (Learn • Listen • Visualize • Practice)

Built by **Team 4NLPians** for the **Build in AI for India** hackathon.

---

## 1. Executive Summary

---

Ekalivan turns a static government-school textbook chapter into a short, localized, animated video lesson — automatically, using AI once at content-creation time and never again at playback time.

**The problem.** Students in Indian government schools live between a home language and English scientific vocabulary, with little material to bridge the gap. Printed textbooks don't help, and hand-made animated lessons are too slow and too expensive to produce at the scale a public school system needs.

**The approach.** A linear, deterministic AI pipeline reads a textbook chapter (PDF), restructures and paces it for a Class 6 reading level, writes a localized narration script, generates speech and subtitles, and renders a finished video. A **Creator** (teacher/publisher) runs this once per chapter; every **Student** afterwards streams the finished video for free, with zero further AI cost.

**Why it's architecturally sound.** Content creation (AI-heavy, expensive, runs once) is fully decoupled from content consumption (free, offline-friendly, runs for every student, every time). This single decision is what makes the product viable at underfunded public-school scale — cost scales with the number of chapters produced, not the number of students watching.

**Current state.** The system is a working end-to-end product, not a mockup: a Creator can upload a PDF, watch it move through an 8-stage AI pipeline, and publish a real streamable MP4; a Student can pick a medium, class, and subject and watch the resulting lesson. It was shortlisted in the Build in AI for India hackathon.

---

## 2. Product Vision & Design Principles

---

**Vision.** Give every child in India access to high-quality conceptual video lessons, regardless of region or income.

**Mission.** Turn a standard textbook chapter into an engaging, culturally grounded video lesson automatically.

Design principles that shape every decision in the app:

- **Child-first.** Screens and pacing are built around a Class 6 attention span and reading level.
- **Simple over clever.** A predictable, linear pipeline beats a "smart" agent nobody can debug under time pressure.

- **Deterministic.** The same chapter always produces a video of the same structure and quality.
- **Offline-friendly.** Rendered video is cached locally and plays smoothly when the school network drops.
- **Generate once, reuse forever.** All AI cost happens once, during creation — never during playback.
- **Open-source / self-hostable first.** The default LLM path runs locally via Ollama, so no per-view API cost is required.

**Explicitly out of scope (MVP):** a full LMS (grading, assignments), a chat-style AI tutor, adaptive/human tutoring fallback, authored curriculum-management tooling, autonomous web research, and any general-purpose assistant behavior. Ekalivan is a single-purpose textbook-to-video generator, not a chatbot.

---

## 3. Who Uses It — Two Personas

---

The app opens on a **role-select screen** and splits into two portals:

### 3.1 Creator Portal (teacher / publisher)

1. **Login** to the creator portal.
2. **Dashboard** — job metrics (totals, per-status counts, mean completion time) and recent activity.
3. **Upload Book** — submit a textbook chapter PDF plus class level, subject, and chapter title.
4. **Pipeline Progress** — watch the chapter move live through the 8-stage AI pipeline (Textbook Parsing → Curriculum Mapping → Lesson Planning → Teacher Script → Storyboard → Narration (TTS) → Video Rendering → Publishing).
5. **Rendering Progress** — a focused view of the Video Rendering step for one job.
6. **Pipeline Complete** — preview the finished video before it's considered published.
7. **Book / Pipelines / Videos libraries** — browse everything previously uploaded, every job ever run, and every published video.
8. **Profile** — creator account details.

### 3.2 Student Portal (learner)

1. **Splash screen** — branding and "Get Started."
2. **Medium selection** — English or Tamil.
3. **Class (grade) selection** — for the chosen medium.
4. **Subject selection** — for the chosen medium and grade.
5. **Chapter list** — browse chapters available for that medium/grade/subject.
6. **Chapter detail** — watch the generated lesson video with topic breakdown.
7. **Login, Downloads, Profile** — student account, offline/cached video library, and profile management.

The hackathon's real, fully-wired demo path (per the UI roadmap) is: **Creator uploads a Grade 6 Science PDF → AI pipeline runs → Creator previews and publishes → Student opens the app →**

**picks English/Tamil → Grade 6 → Science → Chapter 1 ("The World of Plants") → watches the lesson.** Everything outside that path (other grades/subjects/chapters) may still be placeholder UI.

There is also a standalone, portal-agnostic **generation flow** ( `/generate` , `/videos/:taskId` , `/my-videos` ) inherited from earlier sprints, which starts generation from a fixed demo chapter, polls status, and plays the finished video directly — useful for testing the pipeline independent of either portal.

## 4. The AI Video Generation Pipeline

This is the core of the product: one linear, deterministic pass per chapter, with no branching and no open-ended agent looping. Each stage is a LangGraph node that reads a shared state object, does one job, and hands off.

| # | Stage                  | What it does                                                                                                                        | Underlying tooling                                                 |
|---|------------------------|-------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| 1 | <b>Parser (Intake)</b> | Extracts text/tables from the source PDF into clean Markdown.                                                                       | Microsoft <b>MarkItDown</b>                                        |
| 2 | <b>Curriculum</b>      | Structures the Markdown into concept sections.                                                                                      | LLM call via the LLM Provider Layer                                |
| 3 | <b>Lesson Planning</b> | Paces those sections for a Class 6 lesson (density, ordering).                                                                      | LLM call                                                           |
| 4 | <b>Teacher</b>         | Rewrites sections into localized, student-friendly narration (regional analogies, bridges home-language and scientific vocabulary). | LLM call                                                           |
| 5 | <b>Storyboard</b>      | Turns narration into timed, scene-by-scene visual beats.                                                                            | LLM call                                                           |
| 6 | <b>Narration (TTS)</b> | Synthesizes per-beat audio and captures word-level timestamps for subtitles.                                                        | <b>Edge TTS</b> (Microsoft, network call, no API key)              |
| 7 | <b>Video Rendering</b> | Composites narrated beats into one streaming-ready MP4.                                                                             | <b>MoviePy + FFmpeg</b> , plus a pluggable clip source (see below) |
| 8 | <b>Publishing</b>      | Validates the rendered file, writes a cache-manifest entry, and returns a servable <code>video_url</code> .                         | Local static-asset storage                                         |

**Job lifecycle:** `QUEUED` → `PROCESSING` → `COMPLETED` (or `FAILED` , with an `error_message` ). A render-slot semaphore caps how many jobs run the heavy rendering stage concurrently, so accepting a request ahead of capacity never spawns unbounded MoviePy/FFmpeg work. Clients poll `GET /api/v1/videos/{task_id}` and the backend reports the current stage ( `progress_node` ) as each LangGraph node completes.

**Visual generation options (video clip source).** Rendering isn't limited to solid-color caption cards — the backend can select the source of each beat's visuals via `VIDEO_CLIP_PROVIDER` : - `local` (default) — a local icon-animation library composites a simple animated visual per beat, no network/API cost. - `veo` — beats are generated as real AI video clips through **Google Gemini Veo 3** ( `VeoVideoGenerationTool` ), requiring a `GOOGLE_API_KEY` . - `kaggle` — pre-generated clips are pulled

from a Kaggle-produced clip set ( `KaggleClipTool` ), fed by an offline batch-generation notebook pipeline (see §6).

**LLM resilience.** Every LLM-backed stage goes through a single internal `ILlmProvider` interface — no application code calls a vendor SDK directly. The default provider is a local **Ollama** server; a `FallbackLlmProvider` can chain a primary and a fallback Ollama model so a single model's failure or unavailability doesn't stall the whole pipeline. Swapping or mixing providers is a configuration change, not a code change — this also gives the project a credible offline story for schools with unreliable connectivity to commercial APIs.

**Multilingual note.** Swapping the narration language is a TTS configuration change, not a pipeline change — language is already isolated as a parameter.

---

## 5. Backend API (FastAPI)

---

Base path `/api/v1/videos` :

| Endpoint                    | Purpose                                                                                                                                                                                                                                          |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>POST /generate</code> | Accepts <code>class_level</code> , <code>subject</code> , <code>chapter_title</code> , <code>file_storage_path</code> ; persists a queued job and kicks off the background pipeline; returns <code>202</code> with a UUID <code>task_id</code> . |
| <code>POST /upload</code>   | Accepts a multipart PDF upload, saves it under the server's static-assets tree, and returns the <code>file_storage_path</code> to pass to <code>/generate</code> .                                                                               |
| <code>GET /{task_id}</code> | Current job status, progress stage, and — once available — Markdown content, curriculum sections, storyboard beats, narrated beats, output path, and final <code>video_url</code> .                                                              |
| <code>GET /</code>          | Most recent jobs (lightweight summaries) for dashboard/library views.                                                                                                                                                                            |
| <code>GET /metrics</code>   | Aggregate job counts by status and mean completion time, for the Creator dashboard.                                                                                                                                                              |

Plus system endpoints: `GET /health` (liveness) and static file serving at `/static/**` for rendered video, audio, and uploaded PDFs.

---

## 6. Technology Stack

---

| Layer              | Technology                                                  | Why                                                                                                     |
|--------------------|-------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| <b>Frontend</b>    | Flutter (Dart) + Riverpod + GoRouter + Dio                  | One codebase, smooth UI on low-end Android hardware, compile-safe async state, type-safe named routing. |
| <b>Local cache</b> | <code>shared_preferences</code> , local video/audio caching | Offline playback and session persistence without a heavy embedded DB.                                   |
| <b>Backend</b>     | Python 3.12 + FastAPI (fully async)                         | Native fit with the Python AI/media tooling the pipeline depends on; no blocking call is allowed        |

| Layer                            | Technology                                                                  | Why                                                                                                         |
|----------------------------------|-----------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
|                                  |                                                                             | on the request path.                                                                                        |
| <b>Database</b>                  | PostgreSQL + SQLAlchemy (async) + Alembic migrations                        | Persists job records, status, and pipeline outputs.                                                         |
| <b>AI Orchestration</b>          | LangGraph                                                                   | A fixed, linear state machine — no runaway agent loops, fully replayable/testable per node.                 |
| <b>Document parsing</b>          | Markdown                                                                    | PDF → Markdown extraction without custom parsing code.                                                      |
| <b>LLM inference</b>             | Ollama (local), with a fallback-provider chain                              | Offline-capable, no per-call vendor cost, provider-agnostic via a single interface.                         |
| <b>Text-to-speech</b>            | Edge TTS                                                                    | Free, no API key, word-level timestamps for subtitle generation.                                            |
| <b>Video rendering</b>           | MoviePy + FFmpeg                                                            | Drop-in multi-track composition without a GPU rendering stack.                                              |
| <b>AI video clips (optional)</b> | Google Gemini <b>Veo 3</b> , or pre-rendered <b>Kaggle</b> -generated clips | Pluggable visual source for richer beat animation beyond static caption cards.                              |
| <b>Containerization</b>          | Docker Compose (backend + PostgreSQL)                                       | One-command local dev stack; Ollama runs on the host and is reached via <code>host.docker.internal</code> . |
| <b>Observability</b>             | OpenTelemetry spans → Arize Phoenix collector (per architecture design)     | Latency, token usage, and validation-failure spans on every pipeline node.                                  |

## 7. Architecture at a Glance

The platform is organized into layered responsibilities, top to bottom:

**Presentation (Flutter)** → **API Gateway (FastAPI)** → **AI Orchestration (LangGraph, single shared typed state)** → **AI Agents** (one per pipeline stage, each doing exactly one transformation) → **Skills** (reusable prompt-assembly/validation business logic per agent) → **LLM Provider** (one interface, provider-agnostic) → **MCP-style tool adapters** (Markdown, Edge TTS, MoviePy, FFmpeg, storage — each behind one async contract) → **Infrastructure** (disk, subprocess, network) → **Output Asset** (the finished MP4 + subtitles a student receives).

Key architectural rules enforced throughout:

- **Async-only backend.** No route performs heavy work inline; long-running work (parsing, TTS, rendering) is always dispatched as a background task the moment a request is accepted.
- **Every prompt call is schema-validated.** LLM output is parsed into a Pydantic model; if it doesn't validate, that pipeline stage fails explicitly rather than silently continuing with malformed data.
- **No vendor lock-in.** Application code never imports an LLM vendor SDK directly — always through the provider interface.
- **File-system video cache,** not cloud object storage, keeping the MVP simple and avoiding multi-region latency overhead.

## 8. What's Deliberately Not Built (Yet)

---

Per the project's own YAGNI discipline, these are explicit non-goals for the current version, reserved for a future phase:

- **Phase 2 — Knowledge Retrieval:** a vector store (Qdrant/PGVector) + RAG to pull in supplementary reference material across chapters.
  - **Phase 3 — Conversational Tutor:** moving from a linear pipeline to a cyclic graph so students can pause and ask follow-up questions, with voice-to-voice interaction.
  - No LMS features (grading, assignment tracking), no open-ended chat tutor, no adaptive/human-fallback tutoring, no autonomous web research, and no general-purpose assistant behavior.
- 

## 9. One-Line Summary

---

**Ekalivan is an AI pipeline that converts a textbook chapter PDF into a localized, narrated, subtitled animated video once, and a two-portal Flutter/FastAPI app that lets a teacher produce that video and every student in the class stream it for free, forever.**